Case_study3: mouse skin lipids dataset

Yinyue Zhu

Table of contents

1 Overview

In this case study, we perform feature annotation on a mouse skin lipids dataset. First we process the raw data to extract a feature list, and calculate collision cross section (CCS) values from ion mobility. Then we search the feature list against the CCS Comependium database. Finally we annotate the imzML using METASPACE, and compare METASPACE results with CCS matching results.

The dataset is from Alexander Nyström Lab and available at https://zenodo.org/records/18713016.

2 Data processing in TIMSImaging and CCS calculation

2.1 Introduction

In this case study, we process a MALDI-TIMS-MS1 lipids dataset of mouse skin tissue, extract features with m/z and ion mobility, then convert ion mobility into collision cross section(CCS), and export imzML for METASPACE annotation.

```
import timsimaging

# enable visualization in the Jupyter notebook
from bokeh.io import show, output_notebook
output_notebook()
# disable FutureWarning
import warnings
warnings.filterwarnings('ignore', category=FutureWarning)
```

```
Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): application/javascript, application/vnd.bokehjs_lo
```

```
bruker_d_folder_name = r"D:\dataset\Melanie\Mouse skin lipidomics.d"
dataset = timsimaging.spectrum.MSIDataset(bruker_d_folder_name)
dataset
```

```
MSIDataset with 65052 pixels
        mz range: 299.999-1350.004
        mobility range: 0.700-1.960
```

```
dataset.image()
```

```
Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): application/javascript, application/vnd.bokehjs_ex
```

2.2 Peak processing

First we run the processing workflow to get the feature list. There are 3 columns in the result: m/z, ion mobility and intensity. The ion mobility is in $1/K_0$ (inverse reduced mobility), of which the unit is $V \cdot s \cdot cm^{-2}$. The `process` function computes CCS values by default, here we disable `ccs_calibration` and calculate CCS explicitly later.

```
results = dataset.process(sampling_ratio=0.1, frequency_threshold=0.02, tolerance=3, adaptive
peak_list = results["peak_list"]
peak_list
```

```
Computing mean spectrum...
Traversing graph...
Finding local maxima...
Summarizing...
```

	mz_values	mobility_values	total_intensity
1	302.058416	0.906070	28.807071
2	303.065771	0.907390	273.366334
3	303.065934	1.076893	73.819985
4	303.065516	0.948773	8.603689
5	304.069069	0.906924	3.887010
...	...	...	...
926	1253.899905	1.844601	14.027978
927	1255.913279	1.853024	26.725288
928	1281.929607	1.870141	25.601230
929	1283.943538	1.877011	2.597694
930	1298.554342	1.759565	7.082859

2.3 Convert ion mobility into CCS

However, $1/K_0$ depends on the ion mobility technique and instrument. The data was collected by trapped ion mobility spectrometry (TIMS) while most ion mobility databases are from drift tube (DTIMS), to allow cross-platform comparison, we need to convert ion mobility into CCS, a property only depends on the ion itself.

TIMSImaging converts $1/K_0$ into CCS by a linear model fitted with reference ion mobilities of calibrants. The metadata of calibration is stored in the raw data:

```
dataset.cali_info
```

KeyName	KeyPolarity	Value
CalibrationDateTime	+	2024-07-28T12:37:35+02:00
CalibrationUser	+	unknown
CalibrationSoftware	+	timsTOF
CalibrationSoftwareVersion	+	5.1.8
MzCalibrationMode	+	4
MzStandardDeviationPPM	+	0.209581
ReferenceMassList	+	RedP__0__to__2000 07-03-2024 New List
MzCalibrationSpectrumDescription	+	<unknown>
ReferenceMassPeakNames	+	b'p3\x00p5\x00p9\x00p17\x00p21\x00p25\x00p29\x...
ReferencePeakMasses	+	b'\x17\xd9\xce\xf7S”\x8a@\xc5 \xb0r\xe8\x11\x8...
MeasuredTimesOfFlight	+	b'!\xe8\xa9\xb9\x14\x0c\xf2@e\xaeo\xd5\x7f\xb3...
MeasuredMassPeakIntensities	+	b'\x00\x00\x00\x00\x80<\xc6@\x00\x00\x00\x00\x..
MassesPreviousCalibration	+	b'\xe0\xf4\xb4cY”\x8a@\xd79uE\xec\x11\x8c@\x7f...
MassesCorrectedCalibration	+	b'\x962\xe9BT”\x8a@B\x8e\xd8\xe1\xe7\x11\x8c@\.
MobilityCalibrationDateTime	+	2024-07-27T13:42:36+02:00
MobilityCalibrationUser	+	unknown
MobilityStandardDeviationPercent	+	0.175188
ReferenceMobilityList	+	Tuning Mix ES-TOF CCS compendium (ESI)
CalibrationMobilogramDescription	+	<unknown>
ReferenceMobilityPeakNames	+	b'C6H19N3O6P3\x00C12H19F12N3O6P3\x00C18H19F
ReferencePeakMobilities	+	b'\xbe\xa3\x80$\x12\x90\xe7?\x00\xa0\xefq\x07\...
MeasuredTimsVoltages	+	b':\xe9\xb9o{uY@\x9d\xa9\x10\x8a\x9ada@\x8a:0,...
MeasuredMobilityPeakIntensities	+	b'\x0eJ\x96\x80\x13\xe1CA\xdce\xc5&\xb0\x9feA\..
MobilitiesPreviousCalibration	+	b'\x8a^\x97]\xd1\x9e\xe7?\xfd\xdap2s\xd5\xef?\...
MobilitiesCorrectedCalibration	+	b'\xb5\x1aw\xe9\xb5\x8c\xe7?”d\xaaG\x9a\xc6\xe...
MobilitiyReferencePressure	+	2.195872
MobilitiyPressureCompensationFactor	+	50.000000

Compute CCS. We also have an option to call interal CCS conversion function `tims_oneoverk0_to_ccs_for_mz` from Bruker's TDFSDK API. The TDFSDK is a dependency of AlphaTIMS and would be automatically installed with TIMSImaging. Here we compute CCS values in both methods.

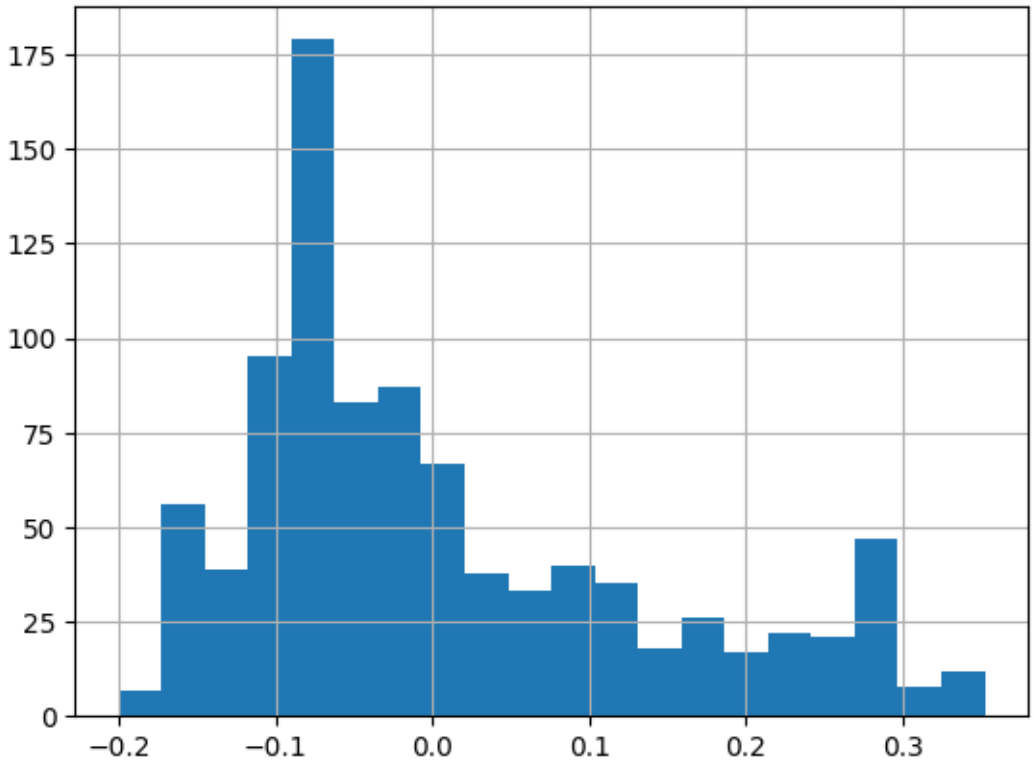
```
ccs_linear_model = dataset.ccs_calibrator(method="linear")
peak_list["CCS_linear"] = ccs_linear_model.transform(peak_list["mz_values"], peak_list["mobi
ccs_bruker_model = dataset.ccs_calibrator(method="internal")
```

```
peak_list["CCS_bruker"] = ccs_bruker_model.transform(peak_list["mz_values"], peak_list["mobi
peak_list["Delta_CCS%"] = (peak_list["CCS_bruker"]-peak_list["CCS_linear"])/peak_list["CCS_b
```

```
peak_list
```

	mz_values	mobility_values	total_intensity	CCS_linear	CCS_bruker	Delta_CCS%
1	302.058416	0.906070	28.807071	189.238376	189.666498	0.225724
2	303.065771	0.907390	273.366334	189.489590	189.916152	0.224605
3	303.065934	1.076893	73.819985	225.188273	225.393028	0.090844
4	303.065516	0.948773	8.603689	198.205047	198.577456	0.187539
5	304.069069	0.906924	3.887010	189.364654	189.791992	0.225161
...	...	...	...	...	...	...
926	1253.899905	1.844601	14.027978	374.204010	373.482918	-0.193072
927	1255.913279	1.853024	26.725288	375.913545	375.181832	-0.195029
928	1281.929607	1.870141	25.601230	379.316539	378.563683	-0.198872
929	1283.943538	1.877011	2.597694	380.709363	379.947853	-0.200425
930	1298.554342	1.759565	7.082859	356.744077	356.131468	-0.172018

```
peak_list["Delta_CCS%"].hist(bins=20)
```



The results of TIMSImaging CCS calculation have very small deviation from Bruker's method.

2.4 Export results

Now we can export the processed data in imzML format for METASPACE annotation.

```
timsimaging.io.export_imzML(dataset, path="Mouse skin lipidomics", peaks=results)
```

3 Lipids annotation with CCS database match

3.1 Introduction

In this part, we search the feature list against CCS databases, then compare with METASPACE annotation results.

```
import pandas as pd
peak_list = pd.read_csv("peak_list.csv")
peak_list
```

	mz_values	mobility_values	total_intensity	ccs_values
0	302.058416	0.906070	28.807071	189.238376
1	303.065771	0.907390	273.366334	189.489590
2	303.065934	1.076893	73.819985	225.188273
3	303.065516	0.948773	8.603689	198.205047
4	304.069069	0.906924	3.887010	189.364654
...	...	...	...	...
925	1253.899905	1.844601	14.027978	374.204010
926	1255.913279	1.853024	26.725288	375.913545
927	1281.929607	1.870141	25.601230	379.316539
928	1283.943538	1.877011	2.597694	380.709363
929	1298.554342	1.759565	7.082859	356.744077

3.2 Annotation with CCS Compendium

First we load the CCS compendium database, which could be downloaded from https://mclean-researchgroup.shinyapps.io/CCS-Compendium
Here we use a subset of +1 charged lipids, which contains 664 adducts with measured drift tube ion mobility (DTIMS) CCS corresponding to 402 unique compounds.

```
df_compendium = pd.read_csv("UnifiedCCSCompendium_FullDataSet_2025-07-28.csv")
compendium_pos_lipids = df_compendium.loc[
    (df_compendium["Super.Class"]=="Lipids and lipid-like molecules")&
    (df_compendium["Charge"]==1)
    ]
compendium_pos_lipids
```

	Compound	Neutral.Formula	CAS	InChi
1045	5-iPF2alpha-VI	C20H34O5	180469-63-0	InChI=1S/C20H34O5/c1-2-3-4-5
1047	8-iso-15(R)-Prostaglandin F2alpha	C20H34O5	214748-65-9	InChI=1S/C20H34O5/c1-2-3-6-9
1049	8-iso-Prostaglandin F2alpha	C20H34O5	27415-26-5	InChI=1S/C20H34O5/c1-2-3-6-9
1051	15(R)-Prostaglandin F2alpha	C20H34O5	37658-84-7	InChI=1S/C20H34O5/c1-2-3-6-9
1053	11-beta-Prostaglandin F2alpha	C20H34O5	38432-87-0	InChI=1S/C20H34O5/c1-2-3-6-9
...	...	...	...	...
3666	Progesterone	C21H30O2	57-83-0	InChI=1S/C21H30O2/c1-13(22)
3668	17-alpha-Hydroxyprogesterone	C21H30O3	68-96-2	InChI=1S/C21H30O3/c1-13(22)
3669	17-alpha-Hydroxyprogesterone	C21H30O3	68-96-2	InChI=1S/C21H30O3/c1-13(22)
3670	d7-Cholesterol Ester (18:1)	C45H71D7O2	1416275-35-8	InChI=1S/C45H78O2/c1-7-8-9-
3671	Cholesteryl Palmitate	C43H76O2	601-34-3	InChI=1S/C43H76O2/c1-7-8-9-

For each feature, we query m/z within a given tolerance, and compare the measured CCS with the reference value.

```
def query_feature(mz, ccs, i, library, ppm_tol=5):

    columns = ['Compound', 'Neutral.Formula', 'CAS',
        'Theoretical.mz', 'Ion.Species', 'Charge',
        'CCS', 'Class']
    # ppm window
    mz_tol = mz * ppm_tol * 1e-6
    mz_min, mz_max = mz - mz_tol, mz + mz_tol

    hit_index =  library['Theoretical.mz'].between(mz_min, mz_max)
    # candidate subset by adduct and mz
    hits = library.loc[hit_index, columns].copy()
    hits["feature_id"] = i
    hits["measured_mz"] = mz
    hits["measured_CCS"] = ccs
    hits["ppm_error"] = (mz - hits["Theoretical.mz"]) / hits["Theoretical.mz"] * 1e6
    hits["ccs_error"] = ccs - hits["CCS"]
```

```
    hits["ccs_dev%"] = ((ccs - hits["CCS"]) / hits["CCS"]).abs() * 100

    if hits.empty:
        return pd.DataFrame()
    return hits
```

Here we set the mass tolerance to 5ppm so that the features get annotated to right formulas. Query each feature, then combine the results into one dataframe:

```
all_results = []
for i, feat in peak_list.iterrows():
    all_results.append(query_feature(feat['mz_values'], feat['ccs_values'], i, compendium_pos

compendium_results = pd.concat([r for r in all_results if not r.empty],
                               ignore_index=True) if any(not r.empty for r in all_results) els
compendium_results
```

	Compound	Neutral.Formula	CAS	Theoretical.mz	Ion.S
0	1,2-Dilauroyl-sn-glycero-3-phosphocholine	C32H64NO8P	18194-25-7	622.4448	[M+
1	Cer 42:02	C42H81NO3	NaN	630.6189	[M+
2	SM 34:01	C39H79N2O6P	NaN	703.5754	[M+
3	PE 34:02	C39H74NO8P	NaN	716.5230	[M+
4	1-Palmitoyl-2-oleoyl-sn-glycero-3-phosphoethan...	C39H76NO8P	26662-94-2	718.5387	[M+
...	...	...	...	...	...
90	SM 42:02	C47H93N2O6P	NaN	835.6669	[M+
91	SM 42:01	C47H95N2O6P	NaN	837.6826	[M+
92	PS 40:05	C46H80NO10P	NaN	838.5598	[M+
93	1,2-Diarachidoyl-sn-glycero-3-phosphocholine	C48H96NO8P	61596-53-0	868.6772	[M+
94	GlcCer 50:04	C56H103NO8	NaN	882.7551	[M+

77 features got 95 hits within 5ppm mass tolerance, while some features have multiple hits:

```
compendium_results[["Neutral.Formula", "Ion.Species"]].value_counts()
```

```
Neutral.Formula  Ion.Species
C41H78NO8P       [M+Na]         4
C42H82NO8P       [M+Na]         3
C39H76NO8P       [M+Na]         2
C40H78NO8P       [M+Na]         2
```

```
C39H76NO8P       [M+H]          2
                                ..
C47H95N2O6P      [M+Na]         1
C48H95N2O6P      [M+H]          1
C48H96NO8P       [M+Na]         1
C48H97N2O6P      [M+H]          1
C56H103NO8       [M+H-2H2O]     1
Name: count, Length: 77, dtype: int64
```

3.3 CCS reduces ambiguity of annotation

Next we inspect features with multiple hits and see if ion mobility can reduce the ambiguity. For example, feature 590 and feature 591 are two isomers at m/z 766.535. Feature 590 is more likely PE(18:1/18:1)(del9-cis), while feature 591 is more likely PE(36:02). Notice there is ambiguity in CCS Compendium itself, that PE(36:02) has multiple isomers with different double bond location and configuration.

```
compendium_results.loc[
    (compendium_results["Neutral.Formula"]=="C41H78NO8P")&
    (compendium_results["Ion.Species"]=="[M+Na]")
]
```

	Compound	Neutral.Formula	CAS	Theoretical.mz	Ion.Species	Charge	CCS	Class
37	PE (18:1/18:1)(del9-cis)	C41H78NO8P	NaN	766.5363	[M+Na]	1	285.8	Glyce
38	PE 36:02	C41H78NO8P	NaN	766.5363	[M+Na]	1	281.0	Glyce
39	PE (18:1/18:1)(del9-cis)	C41H78NO8P	NaN	766.5363	[M+Na]	1	285.8	Glyce
40	PE 36:02	C41H78NO8P	NaN	766.5363	[M+Na]	1	281.0	Glyce

plot ion images for these features. We can see they have different spatial distributions.

```
from bokeh.layouts import row
from bokeh.models import LinearColorMapper
output_notebook()
isomer1, _ = timsimaging.plotting.image(dataset, feature_index=590, results=results)
isomer2, _ = timsimaging.plotting.image(dataset, feature_index=591, results=results)
# adjust color map, high to 50% of maximum intensity
isomer1.select_one(LinearColorMapper).high=0.5
isomer2.select_one(LinearColorMapper).high=0.5
show(row(isomer1, isomer2, width=500, height=500))
```

```
Unable to display output for mime type(s): text/html
```

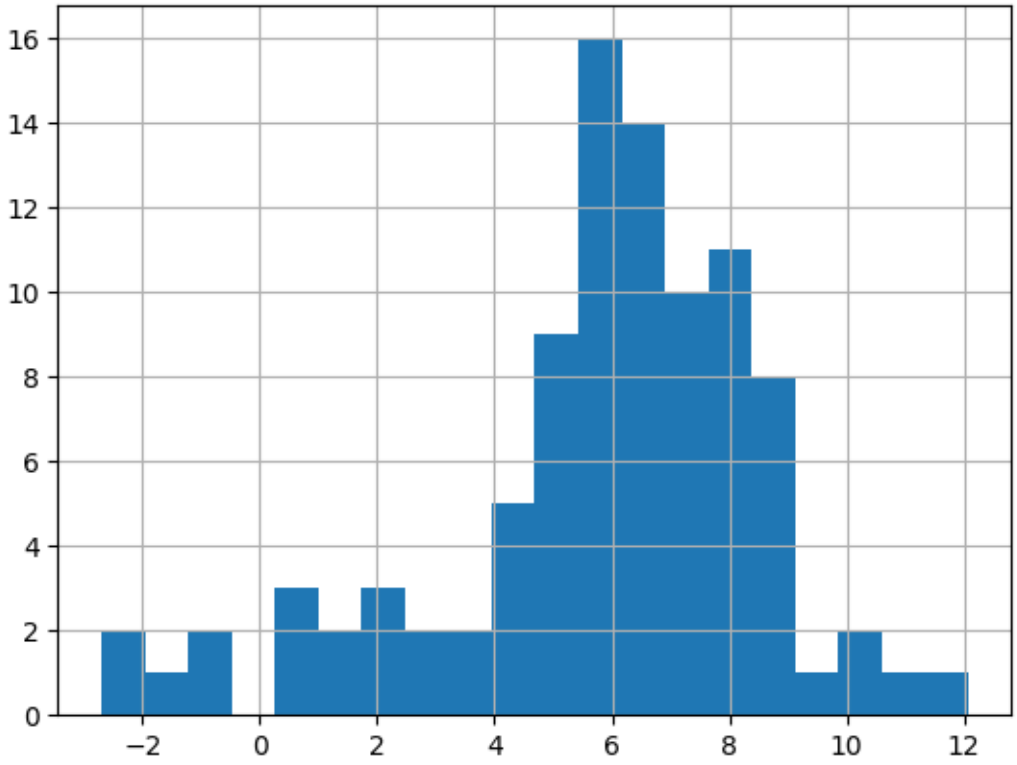
```
Unable to display output for mime type(s): application/javascript, application/vnd.bokehjs_lo
```

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): application/javascript, application/vnd.bokehjs_ex
```

Visualization of CCS error distribution. The CCS errors are not 0-centered, the possible reason could be there is bias between drift tube CCS and TIMS CCS, or the features were matched to the compounds with different ion mobility values, while the true identities are not in the database.

```
compendium_results["ccs_error"].hist(bins=20)
```



3.4 Compare with METASPACE results

The imzML exported in Section 2 was uploaded to METASPACE and annotated without ion mobility. Using the Lipid Maps database with 42k compounds, 96 features were annotated with FDR=5%. The annotation parameters and results are available at https://metaspace2020.org/dataset/2026-01-07_13h02m29s

```
metaspace_results = pd.read_csv("metaspace_annotations.csv", header=2)
print(metaspace_results.shape)
metaspace_results.head(5)
```

```
(96, 24)
```

	group	datasetName	datasetId	formula	adduct	chemMod	ion
0	NaN	MF402upper_noim	2026-01-07_13h02m29s	C40H80NO8P	M+H	NaN	C40H80NO
1	NaN	MF402upper_noim	2026-01-07_13h02m29s	C38H77N2O7P	M+H	NaN	C38H77N2O
2	NaN	MF402upper_noim	2026-01-07_13h02m29s	C41H83N2O6P	M+H	NaN	C41H83N2O
3	NaN	MF402upper_noim	2026-01-07_13h02m29s	C45H91N2O6P	M+H	NaN	C45H91N2O
4	NaN	MF402upper_noim	2026-01-07_13h02m29s	C33H64NO9P	M+H	NaN	C33H64NO

The annotation of METASPACE did not take advantage of ion mobility (CCS), a feature is usually matched with many lipids with the same mass. For example, $C_{42}H_{82}NO_8P$ [M+Na] has 16 candidates in two categories: phosphatidylcholine (PC) and phosphatidylethanolamine (PE).

```
metaspace_results.loc[metaspace_results["formula"]=="C42H82NO8P", "moleculeIds"]
```

```
16    LMGP01010579, LMGP01010581, LMGP01010578, LMGP...
Name: moleculeIds, dtype: object
```

However, the measured CCS (291.5) is closer to PC, implying that the feature is more likely PC than PE.

```
compendium_results.loc[
    (compendium_results["Neutral.Formula"]=="C42H82NO8P")&
    (compendium_results["Ion.Species"]=="[M+Na]")
]
```

	Compound	Neutral.Formula	CAS	Theoretical.mz	Ion.Species	Charge	CCS	Clas
50	PC (18:1(9Z)/16:0)	C42H82NO8P	59491-62-2	782.5676	[M+Na]	1	292.1	Gly
51	PC 34:01	C42H82NO8P	NaN	782.5676	[M+Na]	1	286.0	Gly
52	PE 37:01	C42H82NO8P	NaN	782.5676	[M+Na]	1	285.5	Gly

Next, we compare overall annotations of CCS Compendium and METASAPCE and plot the Venn diagram:

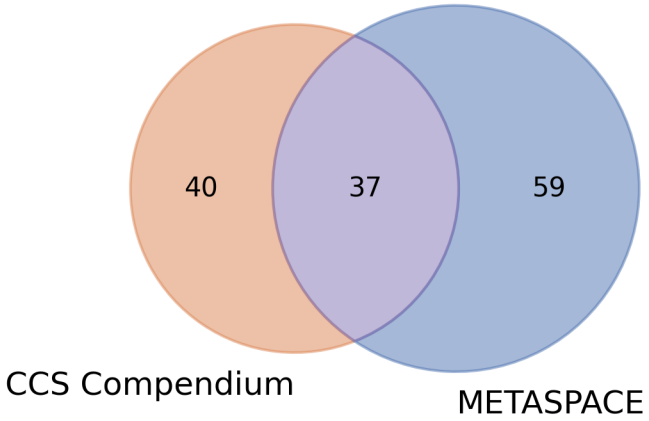
```
import matplotlib.pyplot as plt
from matplotlib_venn import venn2

# format the adduct symbols
metaspace_results['Ion.Species'] = metaspace_results['adduct'].apply(lambda x: '['+x+']')
set1 = set(compendium_results[['Neutral.Formula', 'Ion.Species']].apply(tuple, axis=1))
set2 = set(metaspace_results[['formula', 'Ion.Species']].apply(tuple, axis=1))
plt.figure(figsize=(6,3), dpi=300)

v = venn2(
    [set1, set2],
    set_labels=("CCS Compendium", "METASPACE")
)

# set colors
v.get_patch_by_id('10').set_color('#DD8452')
v.get_patch_by_id('01').set_color('#4C72B0')
v.get_patch_by_id('11').set_color('#8172B2')

for region in ['10', '01', '11']:
    patch = v.get_patch_by_id(region)
    if patch:
        patch.set_alpha(0.5)
#plt.savefig("annotation_venn.png")
plt.show()
```



There are 37 features were annotated as compound+adduct combination by both method. The features only annotated by METASPACE are due to much larger size of database. The features only annotated by CCS Compendium are rare adducts not covered in METASPACE search,‘and possibly with low METASPACE MSM scores.